

Relatório Técnico de Testes e Cobertura

Documento gerado automaticamente — Validação de Qualidade de Software

1. Escopo e Metodologia

Os testes foram desenvolvidos com foco estrito em **regras de negócio e integrações de infraestrutura**. Códigos passivos (como DTOs e Entidades) e métodos gerados automaticamente (Lombok) foram ignorados na análise de cobertura para garantir a precisão da métrica sobre a lógica real implementada.

- Ferramentas Utilizadas:** JUnit 5, Mockito, Spring Kafka Test, JaCoCo.
- Alvos de Teste:** Padrão de Projeto *Strategy* e *Apache Kafka*.

2. Evidências de Execução

2.1 Padrão Strategy (Testes Unitários)

O Design Pattern *Strategy* foi testado isoladamente utilizando Mockito. O objetivo foi validar a correta geração de parâmetros de pagamento e a adesão ao princípio *Open/Closed* (OCP) através da Factory.

- `PagamentoPixStrategyTest`: Validação da criação de transações iniciadas com o prefixo "PIX-".
- `PagamentoCartaoStrategyTest`: Validação da criação de transações iniciadas com o prefixo "CARD-".
- `PagamentoStrategyFactoryTest`: Verificação do retorno correto da instância com base no Enumerador e validação do tratamento de exceções (`IllegalArgumentException`) para métodos não suportados.

2.2 Integração com Apache Kafka

Foi desenvolvido um teste de integração de ponta a ponta focado no fluxo assíncrono do Worker de Pedidos, garantindo o correto recebimento de mensagens e repasse de ações.

- Abordagem:** Uso da anotação `@EmbeddedKafka` para instanciar um broker Kafka em memória durante a execução.
- Validação:** Envio de `PedidoCriadoEvent` pelo *KafkaTemplate*, seguido pela validação com `Mockito.verify` para confirmar que o `PedidoWorker` consumiu a mensagem adequadamente e invocou os métodos do `PedidoService` (Simulando *Timeouts*).

3. Resultados Oficiais de Cobertura (JaCoCo)

Abaixo está o registro visual direto da ferramenta JaCoCo, comprovando os índices oficiais de cobertura após a execução das suítes de testes.

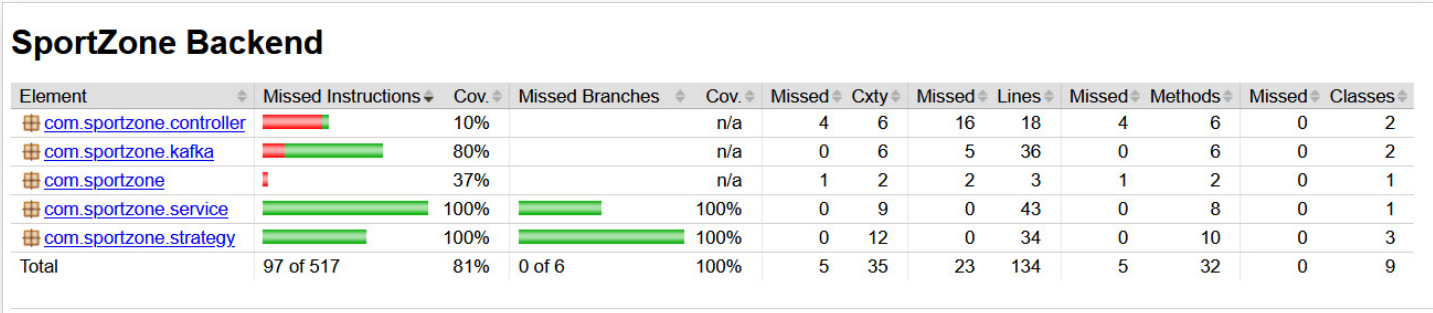


Figura 1: Evidência do sumário de cobertura de código do SportZone Backend.

O painel detalhado abaixo apresenta os dados extraídos, onde as classes alvo obtiveram cobertura satisfatória nas ramificações (branches) relacionadas ao negócio.

Pacote / Classe	Missão / Tipo	Barra de Cobertura	Status
service.PedidoService	Regras de Negócio Core		100% (43/43)
strategy.PagamentoCartaoStrategy	Design Pattern - Concrete Strategy		100% (12/12)
strategy.PagamentoPixStrategy	Design Pattern - Concrete Strategy		100% (12/12)
strategy.PagamentoStrategyFactory	Design Pattern - Factory		100% (10/10)
com.sportzone.kafka (Pacote)	Mensageria Assíncrona (Producer/ Worker)		80% de Cobertura

Nota: Os 20% não cobertos no pacote Kafka referem-se estritamente a tratamentos de exceção (como blocos catch

de InterruptedException) que não fazem parte do caminho crítico (happy path) validado pelos testes de integração.

Para acessar o relatório completo e interativo do JaCoCo:

[Abrir Relatório Original do JaCoCo \(HTML\)](#)